PYTHON LEARNING PROJECT

TaskBook

A single, fully-runnable program that covers nearly every core feature of Python — built around a practical CLI task manager so every concept has a real purpose.

taskbook.py

OVERVIEW SETUP CONSTANTS DECORATOR DATACLASS CLASS FUNCTIONS MAIN LOOP ALL CONCEPTS

00 What the program does

`taskbook.py` is a command-line task manager. You can add tasks with priorities and tags, mark them done, search them, and view statistics. All data is saved to a `tasks.json` file so it persists between runs.

Every feature of the app exists to exercise a real Python concept. Nothing is contrived — the program is genuinely useful, and every language feature earns its place.

VARIABLES & TYPES
Every constant, task field, and menu choice.

DICTS, LISTS, SETS
The menu dispatch table, task list, priority set.

OOP — CLASS
`TaskBook` manages the collection with methods and a property.

OOP — DATACLASS
`Task` uses `@dataclass` for auto-generated boilerplate.

DECORATOR
`@log_errors` wraps methods to catch and display errors cleanly.

GENERATOR
`book.pending()` yields tasks lazily instead of building a list.

COMPREHENSIONS
Filtering, transforming, and serialising task lists in one line.

FILE I/O + JSON
Tasks are saved and loaded with `open()` and `json`.

EXCEPTION HANDLING
Invalid input and corrupt files are caught gracefully.

MATCH / CASE
The main menu uses Python 3.10+ pattern matching.

LAMBDA & SORTED()
Sorting tasks by priority or title with a key function.

STANDARD LIBRARY
pathlib, datetime, functools, collections all appear naturally.

01 How to run it

You only need Python 3.10+ (for `match/case`). No external packages are required — the whole program uses the standard library.

```
$ python --version # confirm Python 3.10+
$ python taskbook.py # run the program

# On first run, it creates tasks.json in the same directory.
# Type 'a' to add a task. Type 'q' to quit.
```

> *If you're on Python 3.9 or earlier, the `match/case` block in `main()` can be replaced with a standard `if/elif` chain — everything else will work unchanged.*

02 Constants & module-level setup

The top of the file defines the program's fixed values. These run once when the module is imported. Notice the three different collection types being used for three different purposes:

```
# pathlib.Path is an object — smarter than a plain string
              Path "tasks.json"

# set — unordered, unique values. Perfect for "valid choices".
               "low"  "medium"  "high"

# dict — maps priority name → display symbol

    "low"     "○"
    "medium"  "◐"
    "high"    "●"


# tuple of pairs — immutable, fixed data, good for (label, value) tables

     "green"   "\033[92m"
     "reset"   "\033[0m"
    # ...


# dict comprehension builds a lookup from the tuple above
# {name: code for name, code in COLOURS}  →  {"green": "\033[92m", ...}
                for            in
```

> **Naming convention:** `UPPER_CASE` *signals "treat this as a constant." Python doesn't enforce immutability at the language level — it's a social contract enforced by convention and linters.*

SET {}	Unordered, unique values. Use for fast membership tests (`if x in PRIORITIES`).
DICT {}	Key → value mapping. O(1) lookup. Keys must be hashable.
TUPLE ()	Immutable sequence. Use when data shouldn't change.
LIST []	Mutable ordered sequence. Use when you need to add/remove.

03 The decorator

A decorator is a function that wraps another function to add behaviour before or after it runs — without modifying the original function's code. In this program, `@log_errors` catches exceptions from any method it wraps and prints them cleanly, so the program never crashes on bad user input.

```
def log_errors
    """The decorator factory — receives the function to wrap."""
              wraps          # copies func's name/docstring onto wrapper
    def wrapper
        # *args  = extra positional args bundled into a tuple
        # **kwargs = extra keyword args bundled into a dict
        try
            return func                      # call the original function
        except  ValueError  KeyError  as
            print f"  Error: {e}"
    return                      # return the wrapper, not the result

# Applied with @ syntax — exactly equivalent to:  add = log_errors(add)
 log_errors
def add self
```

HOW IT FLOWS

1. `@log_errors` above `def add` replaces `add` with `wrapper`.
2. When you call `book.add(...)`, you're actually calling `wrapper`.
3. `wrapper` tries to run the original `add` inside a try/except.
4. If an exception is raised, it's caught and printed — no crash.
5. `functools.wraps` keeps the original function's name in tracebacks.

WITHOUT THE DECORATOR

```
# Every method would need this boilerplate:
def add self
    try
        # actual logic

    except  ValueError  KeyError  as
        print f"Error: {e}"

def remove self
    try
        # actual logic

    except  ValueError  KeyError  as
        print f"Error: {e}"
```

04 The Task dataclass

`@dataclass` is a decorator from the standard library that automatically generates `__init__`, `__repr__`, and `__eq__` from your field annotations. It's the modern, concise way to define data-holding classes.

```
from            import


 dataclass
class Task
                                  # required field — no default
                     "medium"   # optional field — has default
                     False
                     field
    # ↑ IMPORTANT: never write  tags: list = []
    #   That would share ONE list across all Task instances!
    #   field(default_factory=list) creates a fresh [] for each task.

                    field
                        lambda                   now   strftime "%Y-%m-%d"
        # lambda = anonymous single-expression function
        # called at Task() creation time to capture the current date
```

METHODS ON THE DATACLASS

```
    def to_dict self
        # vars(self) returns the instance's __dict__
        return vars self

     classmethod                         # receives the class, not an instance
    def from_dict cls                 "Task"
        return cls                          # ** unpacks dict as keyword args

    def __str__ self
        # __str__ is called by print() and str()
                 f"✓" if self      else f"·"   # inline ternary
                  " " join f"#{t}" for   in self        # generator expr
        return f"{status} {self.title} {tag_str}"
```

> **The mutable default gotcha** *is one of the most common Python bugs.* `tags: list = []` *creates one list object shared by every instance — appending to one task's tags would append to all of them.* `field(default_factory=list)` *creates a new* `[]` *for each instance. Always use* `field(default_factory=...)` *for lists and dicts.*

05 The TaskBook class

`TaskBook` is a traditional class (not a dataclass) because it needs custom logic in `__init__`, a computed property, and a generator method. It wraps the list of tasks and exposes clean methods for every operation.

__INIT__ AND INSTANCE VARIABLES

```
class TaskBook
                         0   # class variable — shared by ALL instances

    def __init__ self            Path
        self                        # instance variable — unique to this object
        self           Task       # empty list, typed for IDE support
        TaskBook                    1
        self _load                 # _ prefix = "private by convention"
```

@PROPERTY — COMPUTED ATTRIBUTES

```
     property
    def stats self
        # @property lets callers write  book.stats  instead of  book.stats()
        # It looks like an attribute but runs code every time.
                len self
                sum 1 for   in self       if          # generator expression
                      Counter            for   in self
        return  "total"        "done"        "pending"
```

GENERATORS — LAZY ITERATION WITH YIELD

```
    def pending self
        # 'yield' makes this a generator function.
        # It produces values one at a time — nothing is computed until
        # the caller asks for the next item.
        for      in self
            if not
                yield

    # Or as a generator expression (one-liner form):
    def by_priority self
        return    for   in self       if
```

FILE I/O WITH CONTEXT MANAGER

```
    def _save self     None
        # 'with open(...) as f' is a context manager.
        # It guarantees the file is closed even if an exception is raised.
        with open self           "w"           "utf-8"  as
            # list comprehension serialises each Task to a plain dict
                 dump    to_dict    for   in self                  2

    def _load self     None
        if not self          exists     # pathlib method — cleaner than os.path
            return
        try
            with open self            "r"           "utf-8"  as
                            load
            self          Task from_dict    for   in
        except      JSONDecodeError
            print "Warning: tasks.json is corrupt — starting fresh."
            self
```

SORTING WITH LAMBDA

```
    def sorted_by self             "priority"         Task
                         "high"  0  "medium"  1  "low"  2
        if        "priority"
            # sorted() returns a NEW list (non-destructive)
            # key= receives a function called on each element to produce the sort key
            # lambda t: ... is an anonymous function: "given task t, return..."
            return sorted self
                             lambda                 get            99
        elif        "title"
            return sorted self          lambda          lower
```

06 Standalone functions

Not everything belongs in a class. Pure functions — functions with no side effects that depend only on their arguments — are easier to test and reuse. The display and input functions are kept separate from `TaskBook` deliberately.

DEFAULT ARGUMENTS & *ARGS / **KWARGS

```
def display_tasks            Task                 "Tasks"     None
    # 'heading' has a default — callers can omit it
    # enumerate(tasks, start=1) gives (1, task), (2, task), ...
    for         in enumerate              1
        print f"  {i}. {task}"

def parse_tags
    # List comprehension with filter and transform in one expression:
    # [expression  for item in iterable  if condition]
    return    strip   lower   for   in     split ","  if   strip

def prompt                             ""
    # Walrus operator (:=) assigns and uses in one expression
           f" [{default}]" if         else ""
          input f"  {message}{hint}: "  strip
    return     or          # 'or default' returns default if raw is empty
```

> `or` **as a default fallback:** `raw or default` *returns* `default` *when* `raw` *is any falsy value — empty string,* `None`*,* `0`*, etc. This is extremely common Python idiom. The longer version would be* `default if not raw else raw`*.*

07 The main loop & dispatch table

The `main()` function runs the interactive loop. Two important Python patterns appear here: the **dispatch table** (a dict of functions) and **match/case** pattern matching.

DISPATCH TABLE — FUNCTIONS AS VALUES

```
# Functions are first-class objects in Python.
# They can be stored in a dict just like any other value.

    "a"
    "l"
    "d"
    "x"
    # ...


# Call the right handler based on user input — no long if/elif chain
                               # look up function by key
handler                        # call it with the TaskBook
```

MATCH / CASE — STRUCTURAL PATTERN MATCHING

```
match
    case "q"
        break                             # exit the while loop

    case ""   "?"
        continue                          # | = OR — match either

    case     if     in
        # guard clause — only matches if the condition is True
        dispatch

    case
        # _ is the wildcard — matches anything not caught above
        print f"Unknown command '{choice}'"
```

THE SCRIPT GUARD

```
# When Python runs a file directly, __name__ == "__main__"
# When the file is imported as a module, __name__ is the module name.
# This guard ensures main() only runs when executed directly.

if             "__main__"
    main
```

08 All concepts — where to find them

Every concept is annotated with a comment in the source file. Use this table as a navigation guide when reading `taskbook.py`.

VARIABLES & TYPES	Section 2 — `DATA_FILE`, `PRIORITIES`, `C` colour dict
F-STRINGS	Throughout — e.g. `f"Task #{i}: {task.title}"`
LIST / DICT / SET / TUPLE	Section 2 — constants block; TaskBook.tasks (list); dispatch (dict)
COMPREHENSIONS	`parse_tags`, `_save`, `search`, `clear_done`, `stats`
GENERATOR / YIELD	`TaskBook.pending()` and `by_priority()`
IF / ELIF / ELSE	Every handler function; `sorted_by`; `_load`
FOR / WHILE	`display_tasks` (for); `main()` (while True)
MATCH / CASE	`main()` — the menu dispatch block
FUNCTIONS + DEFAULTS	`display_tasks`, `prompt`, `parse_tags`
*ARGS / **KWARGS	`log_errors` wrapper; `Task.from_dict` uses `**data`
LAMBDA	`sorted_by` sort keys; `Task.created` default_factory
DECORATOR	`log_errors` — Section 3; applied with `@log_errors`
DATACLASS	`Task` class — Section 4
CLASS + __INIT__	`TaskBook` — Section 5
@PROPERTY	`TaskBook.stats` and `TaskBook.count`
@CLASSMETHOD	`Task.from_dict`
EXCEPTION HANDLING	`log_errors`; `_load`; `run_complete` input parsing
RAISE	`TaskBook.add` — validation logic
FILE I/O + WITH	`_save` and `_load`
JSON	`json.dump` / `json.load` in `_save` / `_load`
PATHLIB	`DATA_FILE = Path(...)`; `filepath.exists()`
DATETIME	`Task.created` default_factory
COLLECTIONS.COUNTER	`TaskBook.stats` — counting tags and priorities
FUNCTOOLS.WRAPS	`log_errors` decorator
TYPE HINTS	Every function signature; `Optional[list]` in `add()`
__STR__ / __REPR__	`Task.__str__` and `Task.__repr__`
IF __NAME__ == "__MAIN__"	Bottom of the file — script entry point guard

> **Suggested learning path:** *Run the program first to see it work. Then read* `taskbook.py` *from top to bottom — every section is labelled and every non-obvious line has a comment. Try making a small change at each step: add a new priority level, add a new command, change the display format. Breaking things and fixing them is the fastest way to learn.*

TASKBOOK PYTHON WALKTHROUGH — CLAUDE 2026